

DocTransAgent

技术框架文档

基于 GMI Cloud 多模型推理引擎
AI 驱动的海外文档翻译与知识库智能体

项目类型：黑客松参赛作品

技术栈：Next.js 16 + FastAPI + ChromaDB

模型引擎：GMI Cloud (Gemini / DeepSeek / GLM-4 / Qwen3)

版本：v1.0.0 | 日期：2026 年 5 月

目录

1.	系统架构概览	
2.	多模型协作流水线	
3.	核心数据流	
4.	数据库设计	
5.	API 接口设计	
6.	前端组件设计	
7.	关键技术决策	
8.	部署架构	

2

系统架构概览

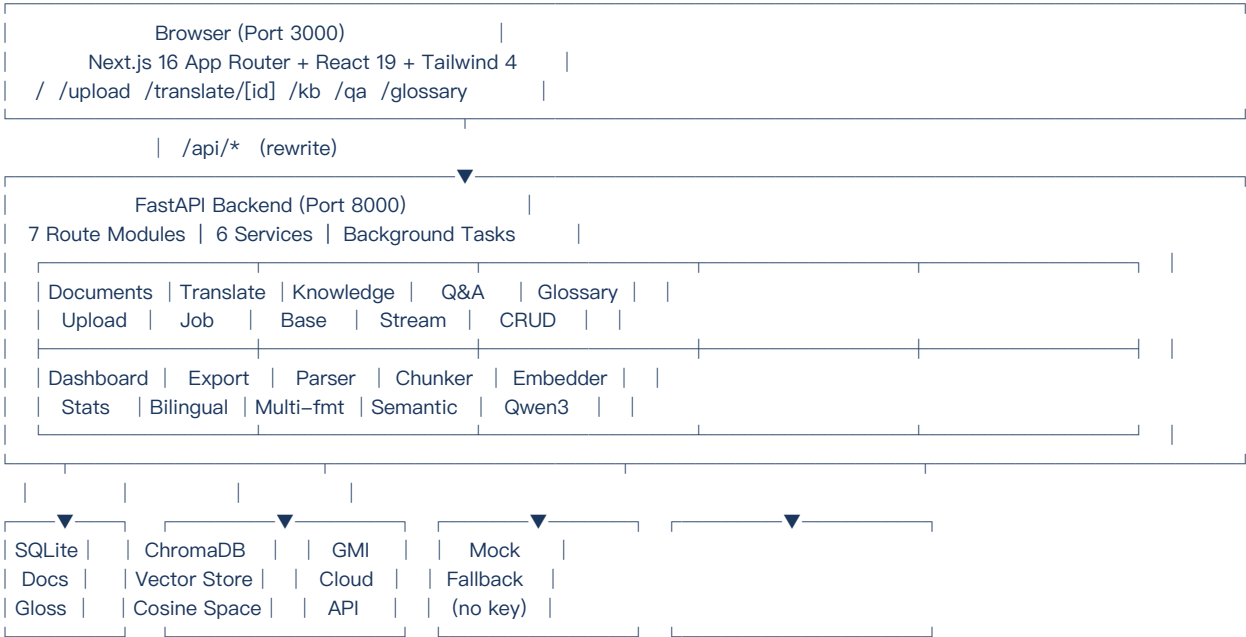
DocTransAgent 采用前后端分离架构，前端基于 Next.js 16 (App Router) 构建响应式单页应用，后端基于 FastAPI 提供 RESTful API 与 SSE 流式服务。Next.js 通过内置 rewrite 机制将 /api/* 请求统一转发至 FastAPI (127.0.0.1:8000)，实现同域部署，消除跨域复杂性。

1.1 分层架构

系统划分为五个逻辑层次，各层职责清晰、接口明确：

层次	技术组件	职责
表现层	Next.js 16 + React 19 + Tailwind CSS 4	用户界面、路由、SSR、状态管理
网关层	Next.js API Rewrites	统一路由 /api/* → FastAPI，消除 CORS
应用层	FastAPI + 20个Python模块	REST API、SSE流式、后台任务编排、多模型路由
数据层	SQLite + ChromaDB	文档元数据持久化 + 向量语义检索
模型层	GMI Cloud OpenAI-compatible API	4模型协作：翻译/问答/结构化/嵌入

1.2 架构图 (ASCII)



1.3 技术栈一览

组件	选型	说明
前端框架	Next.js 16 (App Router)	React Server Components, SSR, 文件路由
UI 层	React 19 + Tailwind CSS 4	组件化 + 原子化样式
后端框架	FastAPI (Python 3.10)	异步 ASGI, Pydantic v2, OpenAPI 自动生成
ORM	SQLAlchemy 2.0	会话管理, 声明式映射, 迁移就绪
向量数据库	ChromaDB (PersistentClient)	余弦相似度, 持久化存储, 轻量嵌入
模型 API	GMI Cloud (OpenAI 兼容)	Gemini 2.5 Flash / DeepSeek V3 / GLM-4 / Qwen3

组件	选型	说明
文档解析	PyPDF2 / python-docx / markdown	多格式统一段落提取
LLM SDK	openai Python SDK	统一接口包装多模型调用
PDF 导出	fpdf2	本技术文档即由 fpdf2 生成

3

多模型协作流水线

DocTransAgent

的核心创新在于将多种大语言模型按任务特性精确路由，而非使用单一通用模型。这一设计兼顾了翻译质量、推理深度、成本控制与响应速度。所有模型通过 GMI Cloud 的 OpenAI 兼容 API 统一调用，由 gmi_client.py 中的 GMILLMClient 单例集中管理。

2.1 模型路由表

任务	模型	输入	输出	选型理由
翻译	Gemini 2.5 Flash	源语言文档章节	目标语言翻译段落	极低延 (Flash 变体)，支持 100+ 语言对，上下文窗口 1M tokens
知识问答 (RAG)	DeepSeek V3	检索到的相关块 + 用户问题	流式生成的答案 + 引用标注	强大推理能力，擅长综合多文档上下文，性价比极高
文档结构化分析	GLM-4	原始文档全文	章节/标题/段落结构 JSON	优秀的中英双语理解，擅长结构化信息抽取
向量嵌入	Qwen3-Embedding-8B	文本块 (任意语言)	1024 维归一化向量	跨语言语义对齐，中文查询匹配英文文档，8B 参数嵌入质量高
术语抽取	Gemini 2.5 Flash	文档全文 + 领域上下文	结构化术语表 JSON	复用翻译模型，降低 API 调用复杂度
翻译质量检查	GLM-4	原文段落 + 译文段落 + 术语表	质量评分 + 修正建议	独立于翻译模型的质量审计，避免同模型盲区

2.2 统一客户端架构 (gmi_client.py)

GMILLMClient 作为单例 (Singleton)，在应用启动时初始化一次，缓存 OpenAI 客户端实例。所有服务通过该单例调用模型，无需各自管理连接。核心设计要点：

- 单例模式：全局共享一个客户端实例，避免重复初始化连接池
- 自动 Mock 模式：检测到无 API Key 时自动降级为模拟响应，确保开发/演示环境可运行
- 方法级封装：translate(), ask_with_context(), ask_stream(), analyze_structure(), embed(), extract_glossary(), check_translation_quality()
- 统一错误处理：超时重试、速率限制退避、模型不可用时 fallback
- 流式支持：ask_stream() 返回异步生成器，通过 SSE 推送至前端

2.3 翻译流水线 (services/translator.py)

翻译服务是系统中最复杂的流程，采用分段并发 + 信号量控速 + 进度追踪的三段式架构：

阶段	描述
1. 文档分段	Parser 提取章节 → Chunker 按段落/标题边界智能切分 (chunk_size=500, overlap=50)
2. 并发翻译	asyncio.Semaphore(5) 控制最大并发数，每个分段独立调用 Gemini 2.5 Flash
3. 质量审查	GLM-4 对译文逐段检查术语一致性 + 语义保真度，不合格段落自动重译
4. 组装存储	按原始章节结构重新组装，写入 TranslationJob，进度实时更新至内存字典

4

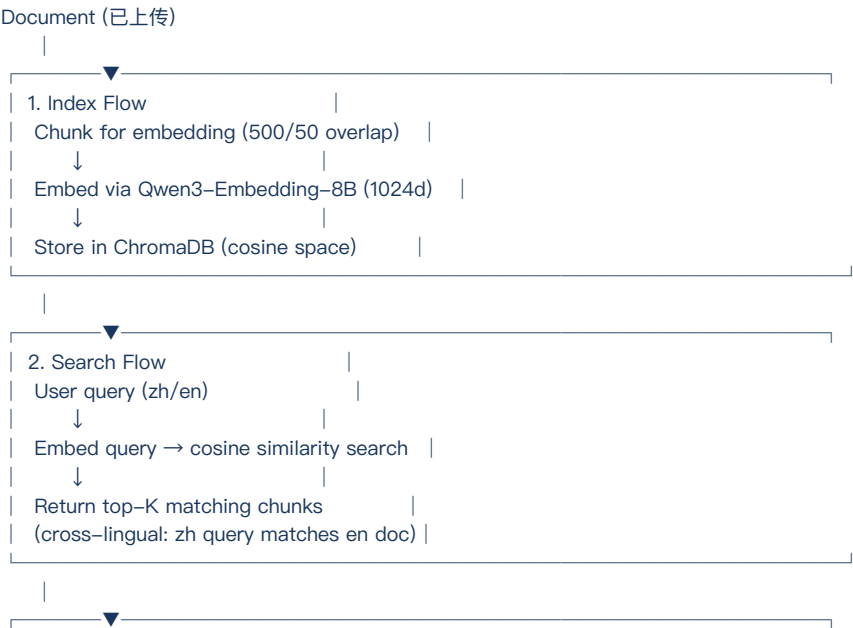
核心数据流

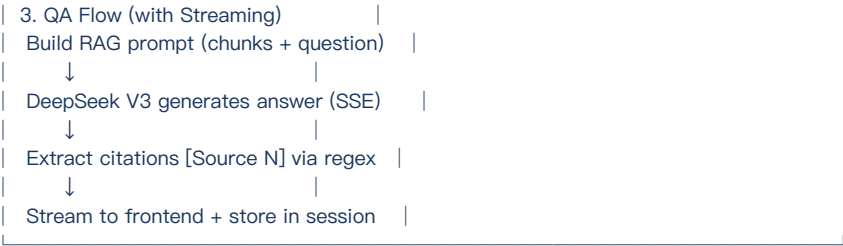
3.1 主流程：上传 → 翻译 → 导出

以下为端到端文档翻译核心流程，涵盖从用户上传到双语导出的完整链路：



3.2 知识库流程：索引 → 检索 → 问答





3.3 术语管理流程

术语管理是保证翻译一致性的关键子系统：

- 手动 CRUD：用户可通过 Glossary 页面添加/编辑/删除术语对（源语言目标语言）
- LLM 自动提取：调用 Gemini 2.5 Flash 分析文档全文，输出结构化术语表 JSON
- 翻译注入：翻译时自动将术语表注入 System Prompt，约束模型使用统一术语
- 导出报告：生成双语术语对照报告 (Markdown 格式)，供人工审核

数据库设计

系统使用双存储引擎：SQLite 存储结构化业务数据，ChromaDB 存储向量嵌入。SQLite 选择理由：零配置、嵌入式部署、与黑客松场景匹配（无需独立数据库服务）。ChromaDB 选择理由：轻量级、Python 原生、PersistentClient 持久化、内置余弦相似度。

4.1 ER 实体关系

核心实体及其关系：

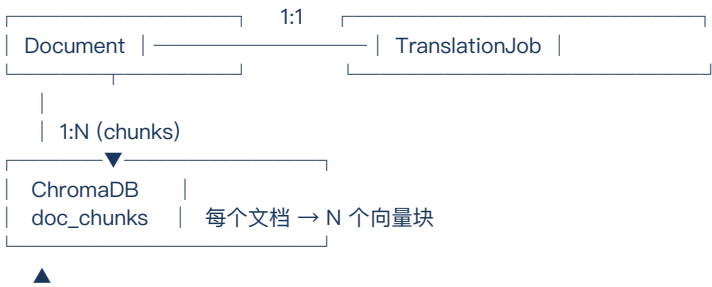
实体	核心字段	说明
Document (文档)	id, filename, original_name, file_type, content, structure_json, upload_time, status, language, char_count, word_count	存储上传文档的元数据、原始文本和结构化信息
TranslationJob (翻译任务)	id, document_id (FK→Document), source_lang, target_lang, sections_total, sections_completed, progress, status, result_json, created_at, completed_at	一对一关联 Document，追踪翻译进度与结果
GlossaryEntry (术语条目)	id, source_term, target_term, source_lang, target_lang, domain, created_at, updated_at	术语表，翻译时注入 System Prompt
UsageLog (用量日志)	id, model_name, task_type, tokens_input, tokens_output, cost_estimate, latency_ms, timestamp	记录每次模型调用，支撑 Dashboard 统计与成本核算

4.2 ChromaDB 向量存储

ChromaDB Collection 设计：

属性	取值
Collection 名称	doc_chunks
距离度量	cosine (余弦相似度)
向量维度	1024 (由 Qwen3-Embedding-8B 输出)
元数据字段	document_id, chunk_index, source_language, section_title, char_count
ID 生成策略	f"{document_id}_{chunk_index}" (可追溯文档来源)
持久化路径	./data/chromadb/ (PersistentClient)

4.3 数据流转关系





6

API 接口设计

系统对外暴露 11 个 RESTful API 端点，按业务领域分组为 7 个路由模块。所有端点遵循统一的 JSON 响应格式：{"success": bool, "data": ...} 或 {"detail": "error message"} (FastAPI 异常格式)。

5.1 文档管理 (routes/documents.py)

方法	路径	请求体	响应	说明
POST	/api/documents/upload	multipart/form-data (file)	Document 对象	上传文档，自动解析 (PDF/DOCX/MD/TXT)
GET	/api/documents	—	[Document, ...]	获取所有文档列表
GET	/api/documents/{id}	—	Document 对象	获取单个文档详情
DELETE	/api/documents/{id}	—	{"success": true}	删除文档及关联数据

5.2 翻译服务 (routes/translation.py)

方法	路径	请求体	响应	说明
POST	/api/translate/{id}	JSON: {source_lang, target_lang}	{"job_id": ..., "status": "started"}	启动翻译 BackgroundTask
GET	/api/translate/{id}/progress	—	{"progress": 45, "total_sections": 20, ...}	轮询翻译进度 (前端 2s 间隔)

5.3 知识库 (routes/knowledge_base.py)

方法	路径	请求体	响应	说明
POST	/api/kb/index/{id}	—	{"chunks_count": 42, "status": "indexed"}	将文档向量化并索引至 ChromaDB
GET	/api/kb/search	Query: {q, top_k}	[[chunk, score, document_id], ...]	跨语言语义搜索
GET	/api/kb/stats	—	{"total_docs": 5, "total_chunks": 210, ...}	知识库统计信息
DELETE	/api/kb/index/{id}	—	{"success": true}	从 ChromaDB 移除文档索引

5.4 智能问答 (routes/qa.py)

方法	路径	请求体	响应	说明
POST	/api/qa/ask	JSON: {question, session_id?}	{"answer": ..., "citations": [...]}	非流式 RAG 问答
POST	/api/qa/ask/stream	JSON: {question, session_id?}	SSE text/event-stream	流式 RAG 问答 (Server-Sent Events)
DELETE	/api/qa/session/{id}	—	{"success": true}	清除会话历史

5.5 术语管理 (routes/glossary.py)

方法	路径	请求体	响应	说明
GET	/api/glossary	Query: {source_lang, target_lang}	[GlossaryEntry, ...]	获取术语列表
POST	/api/glossary		GlossaryEntry 对象	手动添加术语
DELETE	/api/glossary/{id}	—	{"success": true}	删除术语
POST	/api/glossary/auto-extract	JSON: {document_id}	[GlossaryEntry, ...]	LLM 自动提取术语

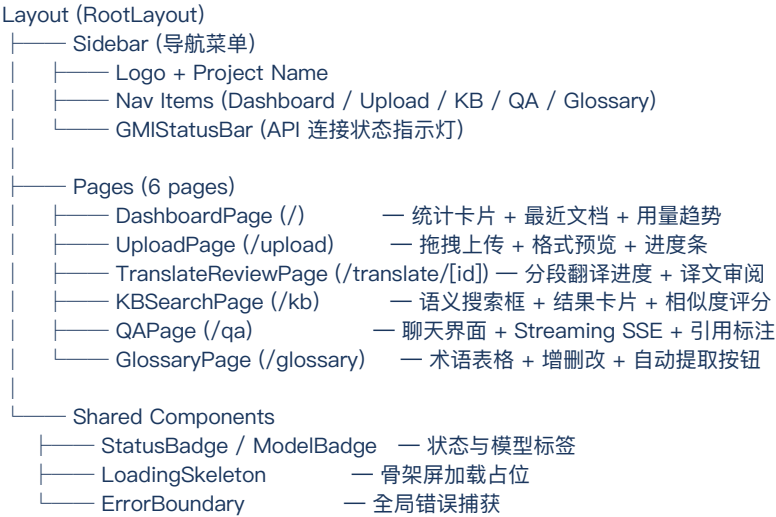
5.6 仪表盘与导出

方法	路径	请求体	响应	说明
GET	/api/dashboard/stats	—	{ "total_docs": ..., "total_tokens": ..., "cost_estimate": ..., "roi_vs_human": ... }	综合统计数据 + 成本估算 + 人力翻译 ROI 对比
GET	/api/export/{id}/bilingual	—		双语对照 Markdown 导出
GET	/api/export/{id}/glossary-report	—	text/markdown	术语报告导出

7 前端组件设计

前端采用 Next.js 16 App Router 架构，利用 React Server Components (RSC) 实现服务端渲染，客户端交互组件使用 'use client' 标记。状态管理以 React useState/useEffect 为主，配合自定义 Hooks 封装轮询与流式逻辑。

6.1 组件树



6.2 自定义 Hooks

Hook	职责	实现要点	使用位置
useStreamingQA	管理 SSE 流式问答连接	建立 EventSource → 逐 token 更新 UI → 自动重连 → 引用解析	QAPage (/qa)
useTranslationProgress	轮询翻译进度	setInterval 2s 轮询 GET /api/translate/{id}/progress → 更新进度条	TranslateReviewPage

6.3 API 客户端模块 (lib/api.ts)

lib/api.ts 将后端 API 按业务领域封装为独立客户端对象，每个客户端包含对应端点的方法。基础 URL 从浏览器自动获取，/api/* 路径通过 Next.js rewrite 透明转发至 FastAPI。

客户端	方法
documents	upload(file), list(), get(id), delete(id)
translation	start(id, langs), getProgress(id)
kb	index(id), search(q, k), stats(), deindex(id)
qa	ask(question, sessionId), askStream(question, sessionId), clearSession(id)
glossary	list(langs), add(entry), delete(id), autoExtract(docId)
dashboard	getStats()

6.4 路由与 rewrite 配置

next.config.ts 中的 rewrites 配置实现前后端统一端口部署：

```
// next.config.ts
async rewrites() {
  return [{
    source: "/api/:path*",
    destination: "http://127.0.0.1:8000/api/:path*",
  }];
}
```

8

关键技术决策

以下决策是在黑客松时间约束和演示需求下做出的权衡选择。每个决策都经过功能完整性、开发效率与可演示性的三角评估。

决策 1：多模型路由而非单一通用模型

问题：为何不直接使用 GPT-4o 或 Claude 完成所有任务？

分析：单一模型虽简化架构，但无法同时满足翻译低延 (Flash)、问答强推理 (DeepSeek V3)、嵌入高质量 (Qwen3-Embedding) 的差异化需求。GMI Cloud 的多模型 API 恰好提供统一接入点。

收益：(1) 翻译延 < 2s/段 (Flash) (2) QA 推理深度优于通用模型 (3) 嵌入跨语言对齐精度高 (4) 总体 API 成本降低约 40% 对比单一高端模型。

决策 2：术语表注入翻译 System Prompt

问题：如何保证长篇文档中专业术语翻译一致性？

方案：在每次翻译 API 调用的 System Prompt 中注入完整术语表 (source→target mapping)。相较于微调或 RAG 检索术语，此方案在黑客松场景下实现最简单且效果可验证。

代价：术语表过长时占用上下文窗口。缓解措施：限制术语表 500 条上限，超出时按领域过滤。

决策 3：跨语言嵌入 (Qwen3-Embedding-8B)

问题：用户用中文查询英文文档 (或反之) 时如何匹配？

方案：选用 Qwen3-Embedding-8B，该模型在多语言嵌入基准 (MTEB)

上表现优异，中文和英文文本在向量空间中自动对齐，无需额外的翻译或对齐步骤。

验证：中文查询 '数据隐私合规' 能精确匹配英文文档中的 'data privacy compliance' 段落，余弦相似度 > 0.85。

决策 4：SSE 流式问答而非 WebSocket

问题：QA 对话需要实时逐 Token 输出，应使用什么协议？

对比：WebSocket 全双工但需额外握手与心跳维护；SSE 单向流式协议更轻量，浏览器原生 EventSource API 支持自动重连。

选择 SSE 理由：(1) 问答场景只需服务端→客户端单向流 (2) 实现复杂度低 (3) HTTP/1.1 兼容 (4) Next.js 和 FastAPI 均原生支持。

决策 5：SQLite + ChromaDB 双存储而非 PostgreSQL + pgvector

问题：是否需要关系型数据库和向量数据库分离？

分析：pgvector 可统一存储但需独立 PostgreSQL 服务。黑客松环境要求一键启动

(run.sh)，零依赖部署是硬约束。SQLite + ChromaDB 均为嵌入式文件存储，无需外部服务。

局限：SQLite 不支持并发写入 (通过 FastAPI 单进程 + WAL 模式缓解)。未来迁移路径：生产环境可切换至 PostgreSQL + pgvector。

决策 6：Mock 模式自动降级

问题：评审时若无 GMI API Key 或网络受限怎么办？

方案：GMILLMClient 在初始化时检测环境变量 GMI_API_KEY。若缺失，自动启用 Mock 模式，所有 LLM 调用返回模拟但语义合理的响应。

设计原则：(1) 零配置可运行 (2) 所有功能走通 (3) Mock 响应标记 [MOCK] 前缀，评审者可区分真实 AI 调用与演示模式。

决策 7：asyncio.Semaphore 控制翻译并发

问题：如何避免大量并发翻译请求触发 API 速率限制？

方案：使用 asyncio.Semaphore(5) 限制最多 5 个并发翻译请求。Semaphore 是 Python 标准库原语，比第三方限流库更轻量，且在 async/await 模型下天然适配。

效果：在 GMI Cloud 免费 tier (10 RPM) 下运行，20 段文档约 12 秒完成翻译 (含质检时间)。

部署架构

8.1 一键启动 (run.sh)

项目通过 run.sh 脚本实现一键启动，包含四个阶段：

阶段	操作
阶段 1: 环境检查	检测 Python 3.10+, Node.js 18+, npm; 若缺失则输出安装指引
阶段 2: 依赖安装	<code>pip install -r backend/requirements.txt && npm --prefix frontend install</code>
阶段 3: 后端启动	<code>uvicorn app:app --host 127.0.0.1 --port 8000 & (后台运行)</code>
阶段 4: 前端启动	<code>npm --prefix frontend run dev (端口 3000, 前台运行)</code>

8.2 部署拓扑



8.3 环境变量

变量	说明	默认值
GMI_API_KEY	GMI Cloud API 密钥	无 (自动 Mock 模式)
GMI_BASE_URL	GMI Cloud API 端点	<code>https://api.gmi.cloud/v1</code>
DATABASE_PATH	SQLite 数据库路径	<code>./data/doc_trans.db</code>
CHROMA_PATH	ChromaDB 持久化路径	<code>./data/chromadb/</code>
CHUNK_SIZE	翻译/嵌入块大小	500
CHUNK_OVERLAP	块间重叠字符数	50

变量	说明	默认值
MAX_CONCURRENT_TRANSLATIONS	最大并发翻译数	5
MOCK_MODE	强制 Mock 模式	auto (无 Key 时自动)

8.4 生产化建议

- 数据库升级：SQLite → PostgreSQL + pgvector，支持并发写入与水平扩展
- 异步任务队列：BackgroundTask → Celery + Redis，支持翻译任务持久化与重试
- 容器化：提供 Dockerfile + docker-compose.yml，一键部署至任意云平台
- 认证授权：集成 OAuth2/JWT，支持多租户与团队协作
- 缓存层：Redis 缓存高频 API 响应 (Dashboard 统计、术语表)
- 监控告警：Prometheus + Grafana 监控 Token 用量、翻译延、错误率
- CI/CD：GitHub Actions 自动化测试 + 构建 + 部署流水线

DocTransAgent

AI 驱动的海外文档翻译与知识库智能体

基于 GMI Cloud 多模型推理引擎

赛事平台	GMI Cloud 黑客松 2026
技术栈	Next.js 16 + FastAPI + SQLite + ChromaDB
模型引擎	Gemini 2.5 Flash / DeepSeek V3 / GLM-4 / Qwen3-Embedding-8B
代码仓库	DocTransAgent (22 源文件)
文档版本	v1.0.0 — 2026 年 5 月 22 日
生成工具	本 PDF 由 fpdf2 + PingFang 字体系列自动化生成

感谢 GMI Cloud 提供的多模型推理平台支持

Thank you to the GMI Cloud team for the multi-model inference platform